

Modelo Adapter

O **Adapter** é um padrão de projeto estrutural. O objetivo principal dele é permitir que objetos com interfaces incompatíveis consigam trabalhar juntos. Ele atua como um "tradutor" ou "adaptador" no mundo do software.

Problema:

Imagine que você está programando o software de um painel de controle industrial moderno. Toda a arquitetura do seu sistema, as funções de alerta e os gráficos na tela foram construídos para ler dados de temperatura em Graus Celsius. A interface padrão do seu sistema exige o método `getTemperaturaCelsius()`.

A fábrica, no entanto, comprou um lote de sensores de um fabricante legado. Porém o software embutido nesses sensores fornece as leituras exclusivamente em Fahrenheit, através de um método chamado `obterLeituraFahrenheit()`.

Ou seja:

- Você não pode alterar o código do sensor legado, pois é uma biblioteca de terceiros ou um firmware fechado.
- Você não deve reescrever todo o seu sistema moderno para aceitar Fahrenheit, pois isso violaria o padrão adotado e causaria retrabalho.

Solução:

Você cria uma classe que herda da interface que o seu sistema moderno conhece (Celsius). Por dentro dessa classe, você guarda uma referência para o sensor antigo (Fahrenheit). Quando o sistema pede a temperatura em Celsius, o Adapter intercepta a chamada, pede a temperatura em Fahrenheit para o sensor antigo, aplica a fórmula matemática de conversão internamente e devolve o valor em Celsius para o sistema.

O seu sistema continua trabalhando normalmente, sem nunca desconfiar que está, na verdade, se comunicando com um hardware legado.

Código Adapter

O código implementa o padrão de projeto Adapter em C++. A principal função desse padrão é fazer com que duas interfaces incompatíveis trabalhem juntas. Neste caso, ela funciona como um conversor de Temperatura de um sensor em Fahrenheit para Celcius.

Ele está dividido em três arquivos: SistemaSensores.h, SistemaSensores.cpp e main.cpp.

Para executar os arquivos existe um Makefile com os comandos make (para compilar e rodar o código) e make clean para deletar o executável após compilação.

1. A Estrutura do Padrão

Neste arquivo de cabeçalho, são definidos os três papéis principais do padrão Adapter:

- **O Alvo (SensorTemperatura - Target):** É a interface que o seu sistema moderno entende. Ela define o método virtual puro getTemperaturaCelsius(). Qualquer sensor novo que você queira usar no sistema precisa seguir esse padrão.
- **O Adaptado (SensorFahrenheitLegado - Adaptee):** É o dispositivo antigo ou de terceiros que você precisa usar, mas que possui uma interface incompatível. Ele não sabe o que é Celsius; ele só possui o método obterLeituraFahrenheit().
- **O Adaptador (AdaptadorSensor - Adapter):** É a peça central. Ele herda de SensorTemperatura (o que significa que o sistema moderno o aceita), mas guarda internamente um ponteiro para o SensorFahrenheitLegado (o dispositivo antigo).

2. A Lógica de Tradução (Arquivo: SistemaSensores.cpp)

Aqui está a mágica da conversão. O adaptador "finge" ser um sensor moderno, mas por baixo dos panos ele delega o trabalho para o sensor antigo e traduz o resultado:

```
double AdaptadorSensor::getTemperaturaCelsius() const {  
    // 1. O adaptador chama o método antigo em Fahrenheit  
    double fahrenheit = sensorLegado_->obterLeituraFahrenheit();  
  
    // 2. Ele faz a conversão matemática dos dados  
    double celsius = (fahrenheit - 32.0) * 5.0 / 9.0;  
  
    // 3. Ele entrega o resultado mastigado no formato que o sistema moderno espera  
    return celsius;  
}
```

O sensor antigo retorna 104.0 Fahrenheit. O adaptador aplica a fórmula que resulta exatamente em 40.0°C.

3. O Fluxo de Execução (Arquivo: main.cpp)

No arquivo principal, vemos como o sistema se beneficia disso:

1. **Função Cliente (exibirNoPainel):** Note que essa função **só aceita** ponteiros do tipo `SensorTemperatura`. Se você tentasse passar o `SensorFahrenheitLegado` diretamente ali, o código nem compilaria.
2. **A Execução:**
 - O código cria o sensor antigo (`sensorAntigo`).
 - Cria o adaptador (`sensorAdaptado`) passando o sensor antigo para ele.
 - Chama `exibirNoPainel(sensorAdaptado)`. O painel acha que está lidando com um sensor Celsius nativo, mas está na verdade conversando com o adaptador.